

Repaso de programación básica

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Julio 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Escribir, compilar y ejecutar un programa corto en C con gcc.
- Resolver problemas pequeños con variables, condicionales, ciclos, funciones y arreglos.
- Seguir la ejecución de un programa con una traza en tabla.

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA2: usar las herramientas del curso (gcc, advertencias del compilador) desde la primera sesión.

Objetivos de aprendizaje II

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, capítulos 1 (*Introduction to C*) y 3 (*Arrays*).

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

Contenido I

- 1 **Presentación del curso**
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

¿Qué estudia el curso? I

- Un programa tiene dos caras: los **datos** y los **algoritmos** que los transforman.
- Este curso se ocupa de la primera: cómo organizar los datos en memoria y cuánto cuesta cada operación sobre ellos.
- Las estructuras del semestre: arreglos, listas, pilas, colas, tablas hash, árboles y montículos.
- El costo no se adivina: se analiza y se demuestra.
- Los lenguajes de trabajo son C y C++; el cierre del semestre muestra el equivalente en Python.

Evaluación I

- La nota del semestre sale de dos parciales, un examen final, las tareas y otros (quices, ejercicios en clase, participación) y un proyecto.
- Los parciales y el examen final se hacen los viernes, en la sesión de tres horas.
- Las tareas y el proyecto son individuales; el proyecto tiene dos entregas.
- Los quices son sobre todo comprobaciones de lectura.
- Todas las actividades cierran en la semana 16; el examen final es el viernes 20 de noviembre.

Horario y herramientas I

- Sesiones: miércoles de 17:00 a 19:00 y viernes de 14:00 a 17:00, salón PL-3.1.
- Compiladores: gcc y g++, siempre con `-Wall -Wextra`.
- Textos guía: Thareja, Kalicharan, Sebesta y CLRS (4.^a edición).

Contenido I

- 1 Presentación del curso
- 2 Un primer problema**
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

El problema I

Lea tres números enteros e imprima el mayor.

¿Entendimos el problema? I

- **Entradas:** tres enteros a , b , c .
- **Salida:** el mayor de los tres.

Ejemplo

Entrada: 4 9 2 $\rightarrow$ Salida: 9

Pregunta

¿Y si la entrada es 7 7 3?

¿Entendimos el problema? Respuesta I

Respuesta

Con 7 7 3 la salida es 7. El empate no cambia nada: se pide el valor mayor, sin importar cuántas veces aparece.

Entradas	tres enteros a , b , c
Salida	el mayor de los tres
Proceso	??

Análisis (completo) I

Entradas	tres enteros a , b , c
Salida	el mayor de los tres
Proceso	suponer que el mayor es a ; si b lo supera, reemplazarlo; si c lo supera, reemplazarlo

Algoritmo

```
leer a, b, c
mayor <- a
si b > mayor entonces mayor <- b
si c > mayor entonces mayor <- c
imprimir mayor
```

Codificación: la estructura mínima en C I

```
/* Estructura minima de un programa en C */  
#include <stdio.h>  
  
int main(void) {  
    printf("Hola, Estructuras de Datos\n");  
    return 0;  
}
```

- `#include <stdio.h>` trae la entrada y salida estándar.
- La ejecución empieza en `main` y termina en su `return`.
- `return 0` le dice al sistema que todo salió bien.

Compilar y ejecutar I

```
$ gcc -Wall -Wextra hola.c -o hola
$ ./hola
Hola, Estructuras de Datos
```

- gcc traduce el fuente a un ejecutable.
- -Wall -Wextra activan las advertencias del compilador. En este curso se compila siempre con ellas.
- Una advertencia casi siempre señala un error que aún no se manifiesta.

Variables, tipos y lectura I

Tipo	Contenido	Formato
int	enteros	%d
double	reales	%f
char	un carácter	%c

```
int n;  
scanf("%d", &n);           /* lee un entero desde el teclado */  
printf("Leimos %d\n", n); /* lo imprime */
```

- El & de scanf se explicará en el tema de punteros; por ahora, siempre va.

Codificación: mayor3.c I

```
/* Lee tres enteros e imprime el mayor */
#include <stdio.h>

int main(void) {
    int a, b, c, mayor;

    scanf("%d %d %d", &a, &b, &c);

    mayor = a;
    if (b > mayor) {
        mayor = b;
    }
    if (c > mayor) {
        mayor = c;
    }

    printf("El mayor es %d\n", mayor);
    return 0;
}
```

Prueba: traza con la entrada 4 9 2 1

Paso	Instrucción	a	b	c	mayor
1	leer a, b, c	4	9	2	—
2	$\text{mayor} = a$	4	9	2	4
3	$\text{¿}b > \text{mayor?}$ sí	4	9	2	9
4	$\text{¿}c > \text{mayor?}$ no	4	9	2	9
5	imprimir	El mayor es 9			

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos**
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

El problema I

Lea n e imprima la suma de los enteros de 1 a n .

¿Entendimos el problema? I

- **Entrada:** un entero n .
- **Salida:** $1 + 2 + \cdots + n$.

Ejemplo

Entrada: 4 $\rightarrow$ Salida: 10 (porque $1 + 2 + 3 + 4 = 10$)

Pregunta

¿Y con $n = 100$? Escribir las cien sumas a mano no es camino: se necesita *repetir* una acción.

El ciclo while I

Cómo funciona

- Se evalúa la condición.
- Si es verdadera, se ejecuta el cuerpo y se vuelve a evaluar.
- Si es falsa, el ciclo termina y el programa sigue después de la llave.

```
while (condicion) {  
    /* cuerpo: se repite mientras la condicion sea verdadera  
    ↪ */  
}
```


Codificación: suma.c I

```
/* Suma los enteros de 1 a n con un ciclo while */
#include <stdio.h>

int main(void) {
    int n, i, suma;

    scanf("%d", &n);

    suma = 0;
    i = 1;
    while (i <= n) {
        suma = suma + i;
        i = i + 1;
    }

    printf("La suma de 1 a %d es %d\n", n, suma);
    return 0;
}
```

Prueba: traza con $n = 4$

Iteración	$i \leq n?$	suma	i
inicio	—	0	1
1	$1 \leq 4$: sí	1	2

Prueba: traza con $n = 4$ (continúa) I

Iteración	$i \leq n?$	suma	i
inicio	—	0	1
1	$1 \leq 4$: sí	1	2
2	$2 \leq 4$: sí	3	3
3	$3 \leq 4$: sí	6	4

Prueba: traza con $n = 4$ (completa) I

Iteración	$i \leq n?$	suma	i
inicio	—	0	1
1	$1 \leq 4$: sí	1	2
2	$2 \leq 4$: sí	3	3
3	$3 \leq 4$: sí	6	4
4	$4 \leq 4$: sí	10	5
5	$5 \leq 4$: no	se sale del ciclo	

Salida

La suma de 1 a 4 es 10

¿Y si $n = 0$? I

Pregunta

¿Qué imprime `suma.c` con la entrada 0?

¿Y si $n = 0$? Respuesta I

Respuesta

Imprime La suma de 1 a 0 es 0. La condición $1 \leq 0$ es falsa desde el inicio, así que el cuerpo no se ejecuta ni una vez y suma conserva su 0 inicial.

El mismo ciclo con for I

```
suma = 0;
for (i = 1; i <= n; i = i + 1) {
    suma = suma + i;
}
```

- for (inicio; condicion; avance) reúne en una línea las tres piezas del while.
- Los dos ciclos son equivalentes; en los ejemplos del curso se prefiere while por su lectura directa.

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos**
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

El problema I

Lea cinco números enteros y diga cuántos son pares.

¿Cómo guardamos cinco números? I

- Con cinco variables a, b, c, d, e el código se repite cinco veces.
- Con mil números la idea se derrumba. Hace falta una sola variable que guarde muchos valores.

Definición (Thareja, cap. 3)

Un arreglo es una colección de elementos del mismo tipo almacenados en posiciones contiguas de memoria. Cada elemento se accede por su índice, que empieza en 0.

¿Cómo guardamos cinco números? II

El arreglo datos con 4 7 10 3 8

datos	4	7	10	3	8
	0	1	2	3	4

datos[0] vale 4 y datos[4] vale 8: cinco casillas, índices de 0 a 4.

Funciones I

Qué es una función

Un bloque con nombre que recibe parámetros, hace un cálculo y retorna un valor. Permite darle nombre a una idea y reutilizarla.

```
/* retorna 1 si x es par, 0 si es impar */  
int es_par(int x) {  
    int resultado;  
  
    resultado = 0;  
    if (x % 2 == 0) {  
        resultado = 1;  
    }  
    return resultado;  
}
```

- % es el operador módulo: el residuo de la división entera. Un número es par cuando su residuo entre 2 es 0.

Codificación: Llenar el arreglo I

```
int main(void) {  
    int datos[5];  
    int i, pares;  
  
    i = 0;  
    while (i < 5) {  
        scanf("%d", &datos[i]);  
        i = i + 1;  
    }
```

- Primer recorrido: se llena el arreglo casilla por casilla, con i de 0 a 4.

Codificación: contar los pares I

```
pares = 0;
i = 0;
while (i < 5) {
    if (es_par(datos[i]) == 1) {
        pares = pares + 1;
    }
    i = i + 1;
}

printf("Hay %d numeros pares\n", pares);
return 0;
}
```

- Segundo recorrido: se le pregunta a cada casilla y se acumula.

Prueba: traza con 4 7 10 3 8 |

<i>i</i>	datos[i]	¿par?	pares
0	4	sí	1
1	7	no	1
2	10	sí	2
3	3	no	2
4	8	sí	3

Salida

Hay 3 numeros pares

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes**
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

Asignar no es comparar I

= no es ==

= asigna; == compara. Confundirlos compila y corre, pero con otro significado.

```
x = 0;  
if (x = 5) {    /* asigna 5 a x; la condicion vale 5: verdadera  
    ↪    */  
    printf("entro al if aunque x era 0\n");  
}
```

- -Wall lo advierte: suggest parentheses around assignment used as truth value. Otra razón para compilar siempre con advertencias.

División entera I

Entre enteros, la división descarta los decimales

7 / 2 vale 3, no 3.5. El resultado de dividir dos int es un int.

```
promedio_mal = (a + b) / 2;    /* con a=7, b=2: da 4.0 */  
promedio_bien = (a + b) / 2.0; /* con a=7, b=2: da 4.5 */
```

- Basta con que un operando sea real (2.0) para que la división conserve los decimales.

Salirse del arreglo I

`datos[5]` no existe

Un arreglo de 5 casillas tiene índices de 0 a 4. Leer o escribir `datos[5]` es comportamiento indefinido: a veces revienta el programa y a veces calla y corrompe otros datos, que es peor.

- C no verifica los límites del arreglo en ejecución; esa responsabilidad es del programa.
- Regla práctica: todo recorrido con `while (i < 5)`, nunca con `while (i <= 5)`.

Variables sin inicializar I

C no pone las variables locales en 0

Si suma no arranca en 0, el ciclo acumula sobre el valor basura que hubiera en esa posición de memoria y el resultado cambia de una ejecución a otra.

```
int suma;           /* valor indefinido */
suma = 0;           /* inicializar antes de acumular */
```

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios**
- 7 Reglas de codificación limpia
- 8 Resumen y referencias

¿Qué imprime este programa? I

```
int i, s;  
  
i = 1;  
s = 0;  
while (i <= 3) {  
    s = s + 2 * i;  
    i = i + 1;  
}  
  
printf("%d\n", s);
```

¿Qué imprime este programa? Respuesta I

Iteración	$i \leq 3?$	s	i
inicio	—	0	1
1	sí	2	2
2	sí	6	3
3	sí	12	4
4	no	se sale del ciclo	

Respuesta

Imprime 12: la suma $2 + 4 + 6$.

Ejercicios con un arreglo de 5 enteros I

Sobre un arreglo datos de 5 enteros leídos del teclado:

- 1 Imprimirlo en orden inverso (del índice 4 al 0).
- 2 Hallar el máximo.
- 3 Calcular el promedio con decimales.

Pistas I

- 1 Recorrer con i desde 4 hasta 0, restando: $i = i - 1$.
- 2 La misma idea de `mayor3.c`: suponer que el máximo es `datos[0]` y reemplazarlo cada vez que otra casilla lo supere.
- 3 Acumular la suma y dividir entre 5.0, no entre 5; la sección de errores comunes explica por qué.

Ejercicio de cierre I

Lea un entero positivo e imprima cuántos dígitos tiene.

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia**
- 8 Resumen y referencias

Las reglas del curso I

En todo código del curso

- Sin `break`, sin `continue`, sin `goto`.
 - Un solo `return`, al final de cada función.
 - Preferir `while` y las construcciones simples.
 - Solo biblioteca estándar; comentarios en español.
 - Compila sin advertencias con `gcc -Wall -Wextra`.
-
- La razón: una función con una sola salida y un ciclo cuya condición cuenta la historia completa se trazan sin sorpresas.
 - Estas reglas se evalúan en tareas, parciales y proyecto.

Un solo return: así no !

```
/* asi no: dos puntos de salida */  
int es_par(int x) {  
    if (x % 2 == 0) {  
        return 1;  
    }  
    return 0;  
}
```

- La función tiene dos puertas de salida; al trazarla hay que vigilar ambas.

Un solo return: así sí!

```
/* asi si: un solo return al final */  
int es_par(int x) {  
    int resultado;  
  
    resultado = 0;  
    if (x % 2 == 0) {  
        resultado = 1;  
    }  
    return resultado;  
}
```

- El resultado se acumula en una variable y la función sale por un único punto, el mismo en todos los casos.

Sin break: así no !

Buscar la posición del primer número negativo en datos, de 5 casillas:

```
/* así no: el ciclo se rompe desde adentro */  
i = 0;  
while (i < 5) {  
    if (datos[i] < 0) {  
        break;  
    }  
    i = i + 1;  
}
```

- ¿Cuándo termina el ciclo? La condición dice una cosa y el break escondido dice otra.

Sin break: así sí l

```
/* así sí: la condición lo dice completo */  
i = 0;  
while (i < 5 && datos[i] >= 0) {  
    i = i + 1;  
}
```

- && es el y lógico: se avanza mientras queden casillas y no haya aparecido un negativo.
- Leer la condición del while basta para saber cuándo termina el ciclo.
- Con `datos = 4, 7, -10, 3, 8` ambas versiones terminan con `i = 2`; la diferencia es cómo se leen.

Contenido I

- 1 Presentación del curso
- 2 Un primer problema
- 3 Ciclos
- 4 Funciones y arreglos
- 5 Errores comunes
- 6 Ejercicios
- 7 Reglas de codificación limpia
- 8 Resumen y referencias**

Ideas clave

- Todo problema se trabaja igual: análisis, diseño, codificación y prueba con una traza.
- Un programa en C se compila con `gcc -Wall -Wextra` y las advertencias se atienden.
- El `while` repite mientras la condición sea verdadera; si es falsa de entrada, el cuerpo no se ejecuta.
- Un arreglo de n casillas se indexa de 0 a $n - 1$.
- Las funciones le dan nombre a un cálculo; en el curso se escriben con un solo `return` y sin `break` ni `continue`.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulos 1 (*Introduction to C*) y 3 (*Arrays*).
-  N. Kalicharan. *Data Structures in C*. 2008. Capítulo 1.